

MargaMetis — Full Advancement Plan

Phase 1 — Spatial Database Integration (top priority)

1.1 — Enable PostGIS on the existing Postgres DB `CREATE EXTENSION postgis`, add `geoalchemy2`, move off the manual `ALTER TABLE` migration hack in [backend/app/init.py:60-70](#) onto real Alembic migrations (you need this scaffolding anyway once geometry columns exist).

1.2 — Deduplicated spatial road-network store New `osm_nodes(id, geom POINT)` / `osm_edges(id, u, v, length_m, highway, geom LINESTRING, attrs JSONB)` tables with GiST indexes. One-time migration ingesting the baked `chennai_central.graphml` and everything currently duplicated across `graph_cache/*.graphml`, deduped by OSM id. Rewrite [route_optimizer/graph/manager.py](#) to pull only the nodes/edges inside `ST_DWithin(...)` of a query's bbox and build a small request-scoped `nx.MultiDiGraph`, instead of one full overlapping GraphML file per query.

1.3 — Spatial nearest-node lookup Replace `ox.distance.nearest_nodes()` — which rebuilds a `BallTree` over the whole loaded graph on every call ([backend/app/routes/route_api.py:278-279](#)) — with `ORDER BY geom <-> point LIMIT 1` against the GiST index.

1.4 — Falls out for free The process-global `optimizer` singleton racing on `.graph` across concurrent requests ([backend/app/routes/route_api.py:22-29](#)) disappears once graph loading is a stateless query returning a small local object per request.

1.5 — Search-history spatial analytics Add `origin_geom/dest_geom` to `SearchHistory` ([backend/app/models.py](#)) for heatmap/cluster queries (`ST_ClusterKMeans`) surfaced on the admin dashboard.

Resume-relevant skills demonstrated: PostGIS, spatial indexing (GiST), SQL query design (`ST_DWithin`, `k-NN <->` operator), schema/migration design (Alembic), reducing redundant external API calls via persistent spatial caching, system design for geospatial data.

Phase 2 — Business/Routing Logic Correctness & Depth

2.1 — Unify the three disagreeing speed models `optimizer.py`'s flat per-vehicle speed ([route_optimizer/optimizer.py:13](#)), `route_api.py`'s inline per-highway speed dict ([backend/app/routes/route_api.py:344-345](#)), and `cost_function.py`'s per-edge `maxspeed` table ([route_optimizer/intelligence/cost_function.py:4-13](#)) currently give **different ETAs for the**

same trip depending on which endpoint is hit. Consolidate to one shared speed model consumed by all three.

2.2 — Move the peak-hour penalty into the cost function Currently applied post-hoc as a flat multiplier on whatever path A* already chose ([backend/app/routes/route_api.py:350](#)) — it should influence edge weights during search so "fastest at 6pm" actually routes differently than "fastest at 3am."

2.3 — Reconcile the two safety/quality tables `cost_function._SAFETY` and `confidence_scorer.ROAD_QUALITY` ([route_optimizer/intelligence/cost_function.py:15-24](#) vs [route_optimizer/confidence_scorer.py:81-98](#)) rank road types on different, disagreeing scales — single source of truth.

2.4 — Fix silent edge-drop in confidence scorer `_extract_edges` uses `edge_data.get(0, {})` instead of the min-cost-parallel-edge selection used everywhere else ([route_optimizer/confidence_scorer.py:297-300](#)).

2.5 — Add turn/maneuver cost modeling Cost function currently only weighs edge length × road type; no penalty for turns/signals, so a numerically-shorter route with 20 turns beats a smoother one even when it's slower in practice.

2.6 — Topological route diversity Yen's k-shortest dedup only compares distance/composite-score proximity ([route_optimizer/intelligence/route_ranker.py:46-59](#)), not actual edge overlap — swap/augment with a Jaccard-overlap check so "3 alternate routes" are genuinely distinct paths.

2.7 — Contradiction resolution in the rule-based fallback `_rule_extract()` has no equivalent to the LLM path's `contradiction_resolution` field ([route_optimizer/intelligence/constraint_engine.py:147-254](#)) — "avoid highways, prefer highways" is silently accepted.

Resume-relevant skills demonstrated: graph algorithms (Dijkstra, A, *bidirectional* A, Yen's k-shortest — already implemented from scratch, a strong signal on its own), cost-function/multi-objective optimization design, NLP-driven constraint extraction with LLM+rule-based fallback.

Phase 3 — App-Layer Hardening

3.1 — Fix privilege escalation (critical, do this regardless of ordering) `role = data.get('role', 'user')` in [backend/app/auth_api.py:11](#) lets any caller self-register as admin.

3.2 — Rate limiting on `/auth/login`, `/auth/register` (Flask-Limiter, backed by the existing Redis client).

3.3 — CSRF protection to match `SESSION_COOKIE_SAMESITE='None'` .

Resume-relevant skills demonstrated: auth/session security, OWASP-aware backend hardening.

Phase 4 — Stretch (only if profiling justifies it)

`pgRouting` (`pgr_dijkstra` / `pgr_astar`) directly in SQL if the pure-Python pathfinding loop itself becomes the bottleneck after Phase 1–2. Costs you the custom multi-objective cost function flexibility — don't start here.